Ingeniería de Software Asistida por IA

Tema 3: Arquitectura de Prompts y Modos de Comunicación

Guía de Estudio Detallada para Estudiantes

La Dicotomía de la Comunicación: ¿Humano o Máquina?

Existe un debate estéril en la comunidad sobre cómo hablarle a la IA: algunos puristas afirman que hablarle como a un humano es un error ineficiente, mientras que otros argumentan que usar "Prompts" hiperestructurados mata la creatividad.

Como Arquitecto de Software, debes entender que **ninguno de los dos extremos es un error; la maestría radica en saber CUÁNDO usar cada estilo**. La comunicación con un LLM tiene dos fases radicalmente distintas: La Ideación (Divergencia) y la Ejecución (Convergencia). Usar el lenguaje incorrecto en la fase equivocada resultará en código inutilizable.

Fase 1: Ideación y Análisis (Hablar como Humanos)

En la fase inicial de un proyecto (Levantamiento de Requisitos, Análisis de Tareas, Diseño de Arquitectura), tu objetivo es transmitir una idea difusa y hacer que la red neuronal conecte conceptos aislados. Aquí, hablar de manera conversacional, como si estuvieras en una reunión con un colega experto, es **estrictamente necesario**.

Ejemplo de Prompt: *"Quiero hacer una app para músicos, pero no sé cómo manejar los cobros recurrentes. ¿Qué me sugieres basándote en cómo lo hace Patreon vs Spotify?"*

El Valor Técnico: Al usar lenguaje fluido, le permites a la IA explorar su inmenso corpus de datos, descubriendo casos límite (*Edge Cases*) y heurísticas en las que no habías pensado. En esta fase, la IA es tu Analista; limitarla con estructuras rígidas ahogaría su capacidad de razonamiento lateral.

Fase 2: Ejecución y Mutación (El Compilador)

Una vez que el plan está trazado y llega el momento de que la IA modifique archivos o escriba código, la conversación coloquial se convierte en un peligro absoluto. Aquí, la máquina deja de ser tu "colega analista" y se convierte en tu **Compilador y Ejecutor**.

Si le hablas como a un humano en esta fase (ej. *"Porfa, métele el login a este archivo"*), la máquina alucinará contextos, ignorará restricciones de seguridad y escribirá "Slop". En la fase de mutación debes comunicarte usando **Arquitectura de Prompts**, aplicando sintaxis estructurada para jerarquizar tus órdenes y anular la creatividad del modelo.

Estructuración de Prompts (Markdown)

Las IAs procesan el texto mediante *Atención*. Al usar sintaxis Markdown, diriges matemáticamente esa atención hacia las reglas inquebrantables de tu petición. Nunca envíes un párrafo gigante; divídelo en jerarquías:

```
# 1. Contexto
Estamos en el Sprint 3. Vas a modificar el archivo `/services/auth.ts`.
Asegúrate de respetar el límite impuesto por `@limit(lines: 100)`.

# 2. Tarea
Implementar la función `createRepeatingTask()` basándote ESTRICTAMENTE en la
interfaz `ITask` previamente definida.

# 3. Restricciones Críticas
- No uses `any` bajo ninguna circunstancia.
- Usa el Patrón Singleton para la conexión a Supabase.
- Aplica Inyección de Dependencias para el validador de fechas.
```

Ingeniería Inversa: La IA como Atacante QA

Una técnica avanzada de Prompting es usar a la IA en tu contra antes de programar. Dado que la máquina tiende a ser complaciente y decir "Sí, tu idea es genial", debes forzarla a buscar fallos.

Prompt de Destrucción:

> *"No escribas código todavía. Actúa como un Ingeniero de QA y un Hacker malicioso. Destruye mi propuesta de base de datos. Encuéntrame 3 fallos lógicos críticos, escenarios de carrera (Race conditions) o problemas de escalabilidad que ocurrirán si implemento este diseño."*

Esta técnica revela debilidades estructurales masivas de forma temprana, ahorrándote semanas de refactorización. Obliga al modelo a usar su corpus no para construir, sino para auditar, lo cual suele arrojar resultados de mucha mayor calidad técnica.